CHAPTER 02 · 학습 노트

느려진 서비스, 어디부터 봐야 할까

『주니어 백엔드 개발자가 반드시 알아야 할 실무 지식』 2장 (17~49쪽) 개념 정리

이 장에서 다루는 내용

1. 처리량과 응답 시간
2. 병목 지점과 확장 (수직·수평)
3. DB 커넥션 풀
4. 서버 캐시
5. 가비지 컬렉터와 메모리 사용
6. 응답 데이터 압축
7. 정적 자원과 브라우저 캐시 / CDN
8. 대기 처리
9. 복습 퀴즈

※ 원문을 옮겨 적은 것이 아니라, 책의 내용을 학습용으로 재구성·요약한 노트입니다. 그림은 개념을 설명하기 위해 새로 그렸습니다.

1. 처리량과 응답 시간

사용자는 "결과가 늦게 뜬다", "타임아웃이 난다"로 성능을 체감한다. 성능에는 네트워크·디스크 속도, 메모리, 단말 CPU 등 여러 지표가 얽혀 있지만, **서버 성능**을 볼 때 가장 중요한 두 지표는 **응답 시간**과 **처리량**이다.

1-1. 응답 시간 (response time)

사용자 요청 하나를 처리하는 데 걸리는 시간. API를 호출한 순간부터 응답(JSON 등)을 모두 받을 때까지의 시간이다.

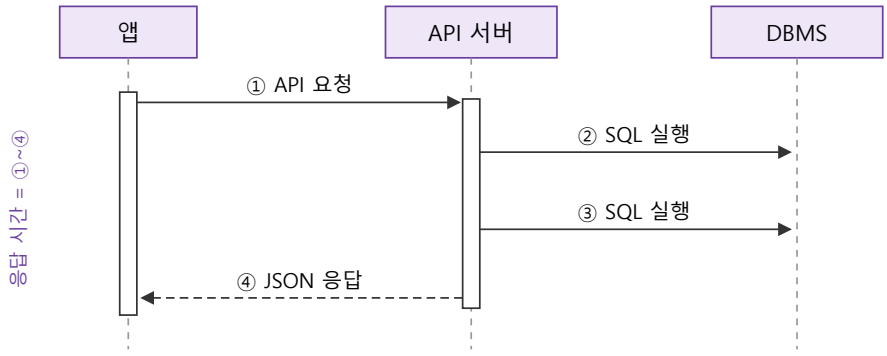


그림 1. API 호출 흐름 — 요청을 보낸 뒤 전체 응답을 받기까지가 응답 시간

응답 시간의 구성

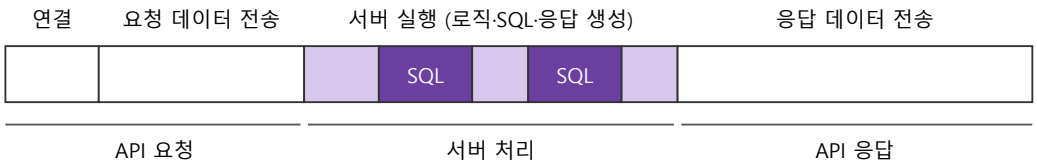


그림 2. 응답 시간 = 요청 전송 + 서버 처리 + 응답 전송

- **요청 단계**: ① TCP로 서버에 연결 → ② 프로토콜(예: HTTP POST + JSON)에 맞춰 데이터를 보냄
- **응답 단계**: 서버가 로직을 실행한 뒤, 요청 때 맺은 소켓으로 응답을 돌려보냄

측정 기준: TTFB vs TTLB

지표	의미
TTFB (Time To First Byte)	응답의 **첫 바이트**가 도착할 때까지 걸린 시간
TTLB (Time To Last Byte)	응답의 **마지막 바이트**가 도착할 때까지 걸린 시간

응답이 작으면 둘의 차이가 거의 없지만, 파일 다운로드처럼 데이터가 크거나 네트워크가 느리면 차이가 커진다. 데이터 특성과 네트워크 환경을 보고 적절한 지표를 골라야 한다. 응답 시간은 대개 1초 미만이라 **밀리초 (ms, 1/1000초)** 단위로 표기한다 (예: 0.025초 = 25ms).

COLUMN · 응답 시간은 왜 중요한가

- 구글의 실험(2009, "Speed Matters"): 검색 지연이 **100ms** 늘면 사용자당 검색 수 **0.2%** 감소, **400ms**면 **0.6%** 감소. 지연을 원상복구한 뒤에도 감소세가 한동안 이어졌다.
- 아마존: 응답 시간이 100ms 늘 때마다 매출이 약 1% 줄어든다는 통계.
- 결론: 빠르게 한다고 매출이 반드시 늘지는 않지만, 느려지면 트래픽·매출에 악영향을 주는 것은 분명하다.

서버 처리 시간의 구성 요소

- 로직 수행 (if, for 등)
- DB 연동 (SQL 실행)
- 외부 API 연동
- 응답 데이터 생성(전송)

이 중 **DB 연동과 외부 API 연동**의 비중이 압도적이다. 저자가 실제 측정한 한 요청의 예:



전체 348ms · 외부 API 53% · 내부 API 13% · DB(SQL 6회) 29% · 로직 17ms(5%)

그림 3. 한 요청의 처리 시간 분해 — API 연동 2건이 66%, DB가 29%, 로직은 5%

외부 API 연동이 없는 서비스라면 보통 **DB 연동이 전체의 70~80% 이상**을 차지한다. 그래서 응답 시간을 줄일 때는 **DB·API 연동 시간**부터 본다.

1-2. 처리량 (throughput)

단위 시간당 시스템이 처리하는 작업량. 주로 **TPS**(Transactions Per Second, 초당 트랜잭션 수)나 **RPS**(Requests Per Second, 초당 요청 수)로 표현하며, 혼용되기도 한다(책에서는 TPS 사용).

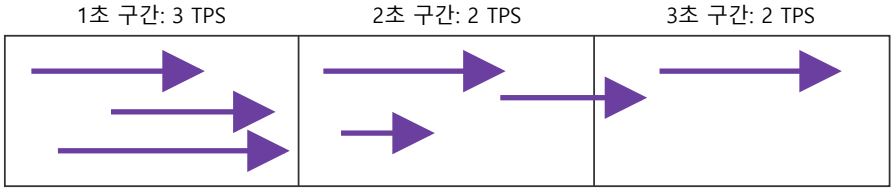


그림 4. 화살표 = 요청(길이 = 응답 시간). 각 1초 구간에서 **완료된** 요청 수가 TPS

최대 TPS와 대기

최대 TPS = 시스템이 처리할 수 있는 최대 요청 수. 동시에 5개를 처리할 수 있고 요청 하나가 1초 걸리면 최대 TPS는 5다. 이를 넘는 요청은 앞 요청이 끝날 때까지 **기다려야** 한다.

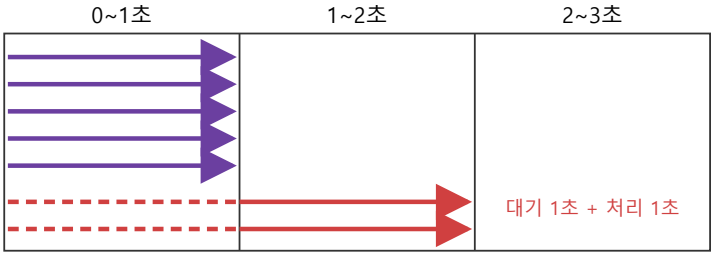


그림 5. 최대 TPS 5인 서버에 7개가 동시에 오면 2개는 응답 시간이 2초가 된다

응답 시간 증가는 사용자 이탈로 이어진다. 대기를 줄이는(= TPS를 높이는) 방법은 두 가지다.

① 동시 처리 수 늘리기
한 번에 처리할 수 있는 요청 수를 늘려 대기 자체를 없앤다.

② 처리 시간 줄이기
요청 하나의 처리 시간을 줄여 같은 시간에 더 많이 처리한다.

먼저 측정하라

"느린 것 같다"는 감으로 이것저것 손대지 말고, 트래픽이 많은 시간대의 **TPS와 응답 시간**을 측정한 뒤 목표치를 정하고 개선안을 도출한다. 가장 쉬운 방법은 **스카우터, 핀포인트, 뉴렐릭** 같은 모니터링 도구로 실시간·과거 TPS를 확인하는 것이다.

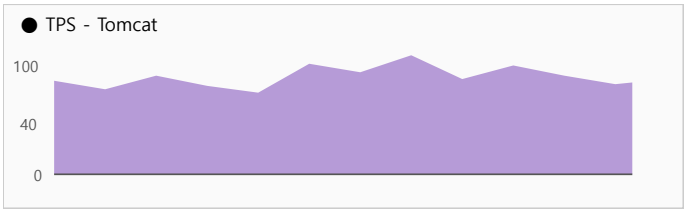


그림 6. 모니터링 도구의 실시간 TPS 그래프 (예시)

MEMO · TPS는 근사치일 수 있다
많은 모니터링 도구는 예컨대 5초 동안 처리한 요청 수를 5로 나누는 식으로 TPS를 근사한다. 정확한 값이 필요하면 **웹 서버 접근 로그**를 엘라스틱서치 같은 곳에 모아 집계하면 되고, 그런 도구가 없어도 접근 로그를 파싱하는 리눅스 명령어와 간단한 코드로 계산할 수 있다.

2. 서버 성능 개선 기초 — 병목 지점과 확장

2-1. 병목 지점

서비스 초기엔 사용자·트래픽·데이터가 작아 문제가 드물다. 규모가 커지면 간헐적 지연이 생기고, 방치하면 큰 장애가 터진다. 전형적인 증상:

- 순간적으로 **모든 요청**이 심하게 느려짐 — 10초 이상 걸리고 연결 시간 초과 오류가 다발
- 서버를 재시작하면 잠깐 괜찮다가 다시 느려지는 현상 반복
- 트래픽이 줄어들 때까지 상황이 계속됨

핵심 원인은 **시스템의 최대 TPS를 넘는 트래픽**이다. TPS를 높이려면 먼저 문제 지점을 찾아야 한다.

- 처리 시간이 오래 걸리는 작업을 식별한다 — 감보다는 **실제 실행 시간 측정**이 우선
- 모니터링 도구의 **실행 시간 추적** 기능으로 어떤 코드가 오래 걸렸는지 찾는다
- 도구가 없다면 의심 코드의 실행 시간을 **로그로라도** 남겨 둔다 → 재발 시 원인 파악에 도움

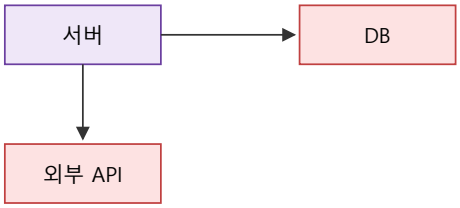


그림 7. 성능 문제는 주로 DB·외부 API 연동 구간에서 생긴다 (DB는 3장, 외부 연동은 4장에서 상세히)

2-2. 수직 확장 (scale-up)

원인을 찾았으면 우선 **급한 불부터 끄고** 근본 해결책을 찾는다. 가장 빠른 응급처치가 수직 확장 — CPU·메모리·디스크 같은 **자원을 키우는 것**이다(더 빠른 CPU, 코어 추가, 메모리 증설, HDD→SSD). 클라우드에서는 특히 빠르게 적용할 수 있다.

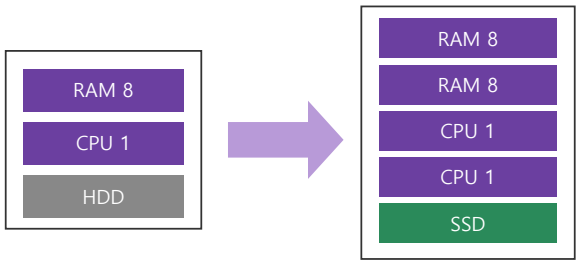


그림 8. 수직 확장 — 장비 한 대의 사양을 올려 응답 시간 단축·처리량 증가

사례 · DB 수직 확장으로 서비스 재개

사용자가 조금만 몰려도 전체가 멈추던 서비스. 원인은 쿼리·테이블 설계 등 DB 쪽에 있었지만 근본 수정엔 시간이 필요했다. 우선 DB 장비의 CPU를 고성능으로 바꾸고 메모리를 늘려 서비스를 재개했다. 완전히 멈추는 것보다 다소 느려도 제공하는 편이 낫고, 근본 해결책을 마련할 **시간도 벌 수 있었다**.

수직 확장은 서버에도 유효하다. 예) 메모리 512MB로 동시 200개를 처리하던 서버에 동시 요청이 250개로 늘면 응답이 느려지기 시작 → 메모리를 1GB로 늘리기만 해도 TPS가 오른다.

한계: 트래픽이 계속 늘면 다시 문제가 생기는데, 매번 수직 확장할 수는 없다. ① **비용**이 크고 ② 장비 한 대의 **용량에도 상한**이 있다.

2-3. 수평 확장 (scale-out)

서버 **대수를 늘려** TPS를 높이는 방법.

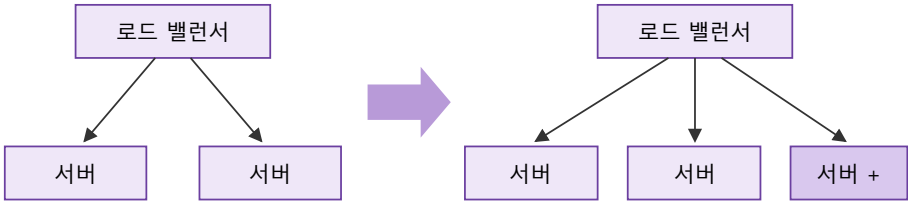


그림 9. 수평 확장 — 서버를 추가해 처리량을 늘린다

알아두기 · 로드 밸런서

서버가 두 대 이상이면 트래픽을 고르게 나눠 주는 **로드 밸런서**가 필요하다.

- **정적 방식** — **라운드 로빈**: 요청을 서버에 차례대로 분배 / **IP 해시**: 클라이언트 IP의 해시값으로 서버 결정 → 같은 클라이언트는 항상 같은 서버로
- **동적 방식** — 서버의 현재 상태를 보고 분배: 연결 수가 적은 서버, 응답 시간이 짧은 서버로 보냄

⚠ 무턱대고 서버를 늘리지 말 것

병목이 DB에 있는데 서버를 늘리면 DB 부하만 커져 **불에 기름을 붓는 격**이 된다. 외부 API가 느린 경우도 마찬가지로 서버 증설로는 TPS가 오르지 않는다. 수평 확장은 **DB·외부 API가 감당 가능한 범위 안에서만** 효과가 있다.

3. DB 커넥션 풀

DB를 쓰려면 ① 연결 → ② 쿼리 실행 → ③ 연결 종료의 3단계를 거친다. 네트워크 연결 생성·종료에는 **0.5초 ~1초 이상** 걸리기도 한다.

예) 쿼리 10ms + 연결/종료 50ms = 60ms → 전체 시간의 80% 이상이 연결·종료에 소모

연결·종료 시간은 응답 시간을 늘리고 처리량을 떨어뜨리며, 매 요청마다 연결하면 트래픽 증가 시 처리량이 **급격히** 떨어진다.

DB 커넥션 풀은 연결된 커넥션을 미리 만들어 보관하고, 애플리케이션이 필요할 때 빌려 쓰고 반환하게 한다. 이미 연결된 커넥션을 재사용하므로 응답 시간이 줄어든다. 사실상 **필수**이며, 스프링 부트는 **HikariCP**를 쓰고 Go 언어는 표준 기능으로 지원한다.

중요 설정 3가지: ① **풀 크기(최소/최대)**, ② **커넥션 대기 시간**, ③ **커넥션 유지 시간(최대 유휴 시간·최대 유지 시간)**

3-1. 커넥션 풀 크기

풀에 미리 만들어 둘 커넥션 수. 서버는 주로 DB와 통신하므로 **가장 중요한 설정**이다.

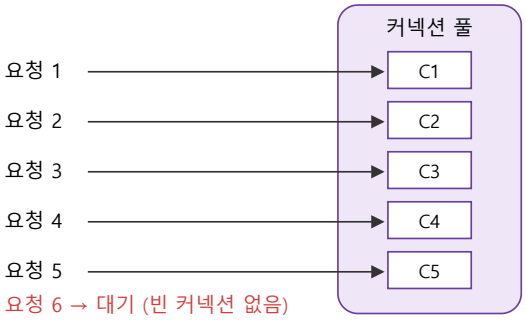


그림 10. 풀 크기 5에 6개가 동시에 오면, 6번째는 커넥션이 반환될 때까지 기다린다

대기를 줄이려면 **응답 시간과 TPS를 함께 고려**해 풀 크기를 정한다.

풀 크기	쿼리 시간	1초 처리량	동시 50개 요청 시
5	0.1초	1 ÷ 0.1 × 5 = **50**	모두 1초 안에 끝남 (최대 0.9초 대기)
5	0.5초	1 ÷ 0.5 × 5 = **10**	전부 처리에 5초 (어떤 요청은 4.5초 대기)
50	0.5초	100	모두 0.5초 안에 처리

최소 크기와 최대 크기

예) 최소 10, 최대 20 → 동시 요청이 10개 이하일 땐 10개를 유지, 그 이상이면 최대 20개까지 늘렸다가, 다시 줄면 커넥션도 점차 줄인다. 은행(낮), 게임(저녁)처럼 트래픽에 시간대 패턴이 있으면 한산할 땐 최소, 붐빌 땐 최대로 운영할 수 있다.

MEMO · 트래픽이 순간 급증한다면

최소 크기를 **최대 크기와 같게** 맞추는 것이 좋다. 점진적 증가에선 연결 생성 시간이 큰 문제가 안 되지만, 급증 시에는 새 연결을 만드는 시간 자체가 성능 저하의 주원인이 될 수 있다.

⚠ 풀 크기를 무턱대고 늘리지 말 것
DB 서버 CPU가 80%에 가까운데 풀을 늘리면 DB 부하가 더 커져 쿼리 시간이 급증한다. 이럴 땐 오히려 풀 크기를 **유지하거나 줄여** DB가 포화되지 않게 해야 한다. 서버를 수평 확장하는 것도 결국 풀 크기를 늘리는 것과 같으니, DB 상태를 확인한 뒤 진행한다.

3-2. 커넥션 대기 시간

풀에 빈 커넥션이 없을 때 기다릴 수 있는 최대 시간. 이 시간 안에 못 얻으면 DB 연결 실패 에러가 난다. 기다린 만큼 응답 시간도 늘어나는데, **HikariCP 기본값은 30초**라 최악의 경우 응답이 30초를 넘길 수 있다.

- 응답 시간이 중요한 서비스는 대기 시간을 짧게 — 보통 **0.5초~3초** 권장
- 짧게 설정하면 풀이 꽉 찼을 때 "일시적 오류" 같은 응답을 빠르게 준다. 오래 무응답인 것보다 **빨리 에러를 주는 편이 낫고**, 서버 부하 증가도 막는다.

예시: 풀 10, 동시 요청 30개, DB 부하로 쿼리가 10초로 늘어난 상황

대기 시간 30초
5초 시점: 10개는 쿼리 실행 중, 20개는 대기 중. 대기자 절반(10명)이 참지 못하고 취소 후 재요청 → 서버는 취소된 작업을 바로 멈추지 않으므로 **대기 30개, 동시 처리 대상 40개**로 증가. 재요청이 반복될수록 부하가 계속 불어난다.

대기 시간 1초
2초 시점: 10개는 쿼리 실행 중, 커넥션을 못 얻은 20개는 이미 **오류 응답**을 받음. 동시 처리 수는 10개로 유지되어 서버 부하가 일정 수준에서 안정된다.



그림 11. 대기 시간이 길면 재요청이 쌓이면서 동시 트래픽이 더 늘어난다

3-3. 최대 유휴 시간, 유효성 검사, 최대 유지 시간

새벽처럼 요청이 거의 없는 시간에는 풀의 커넥션이 오래 놀게 된다. MySQL 등은 일정 시간 상호작용이 없는 연결을 **자동으로 끊는다**(예: 1시간). 끊긴 커넥션을 그대로 쓰면 에러가 나므로 풀은 다음 기능을 제공한다.

설정	내용
최대 유휴 시간	사용되지 않는 커넥션을 풀에 둘 수 있는 최대 시간. 예) 30분 → 30분 넘게 놀던 커넥션은 닫고 제거. **DB의 비활성 연결 유지 시간보다 짧게** 잡으면 DB가 끊기 전에 풀이 먼저 정리한다.
유효성 검사	커넥션이 정상 사용 가능한지 확인. 풀에서 꺼낼 때 또는 주기적으로 검사해 죽은 커넥션을 제거. 실제 쿼리로 검사할 땐 `SELECT 1`, `SELECT 1 FROM dual` 같은 가벼운 쿼리를 쓴다.
최대 유지 시간	커넥션이 생성된 시점부터 살아 있을 수 있는 최대 시간. 예) 4시간 → 유효하더라도 4시간이 지나면 닫고 제거.

MEMO
최대 유휴 시간과 최대 유지 시간은 **무제한으로 두지 말 것**. 풀의 기본값을 확인하고, 무제한이라면 DB 설정을 참고해 적절한 값을 지정한다.

4. 서버 캐시

DB를 수직·수평 확장하면 비용이 크고, 수평 확장해도 **쿼리 실행 시간 자체**는 크게 줄지 않는다. DB를 늘리지 않고 응답 시간과 처리량을 개선하려면 **캐시**를 고려한다. 캐시는 **(키, 값)** 쌍을 저장하는 Map 형태의 저장소로, 자주 조회되는 데이터를 넣어 두면 DB 대신 캐시에서 빠르게 읽어 응답할 수 있다.

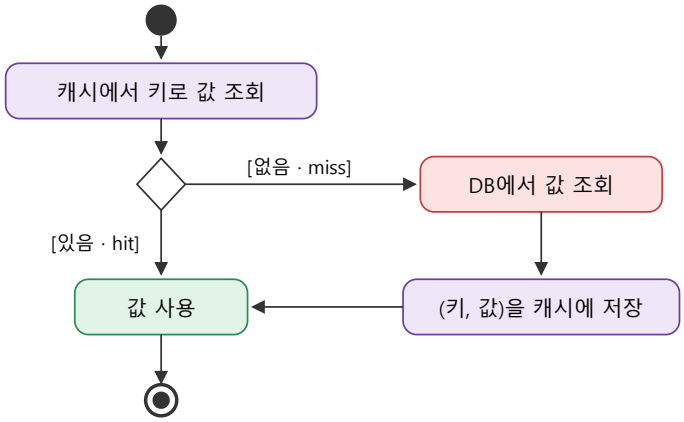


그림 12. 캐시 동작 방식 — 없으면 DB에서 읽어 캐시에 넣고, 다음부터는 캐시에서 바로 사용

DB 조회 결과뿐 아니라 **복잡한 계산 결과**나 **외부 API 연동 결과**도 캐시에 담아 응답 시간을 줄일 수 있다.

> **알아두기 · 캐시 키 선택**
> 저장 대상에 맞는 키를 설계한다. 예) 게시글 상세는 `articles:{번호}`, 최신 인기 글은 `articles:hot10`. 서로 다른 데이터의 **키가 겹치지 않도록** 주의한다.

4-1. 적중률과 삭제 규칙

적중률(hit rate) = 캐시에 존재한 건수 ÷ 캐시 조회 시도 건수

예) 100번 조회 중 87번 존재 → 적중률 0.87(87%). 적중률이 높을수록 DB 연동이 줄어 **응답 시간 감소 · 처리량 증가 · DB 부하 감소**로 이어진다.

가장 단순한 방법은 많이 저장하는 것이다(상품 5,000개 중 100개보다 4,500개를 넣는 편이 적중률이 높고, 전부 넣으면 이론상 100%). 하지만 캐시는 **메모리**를 쓰므로 저장 개수·크기에 한계가 있다(예: 최대 1,000개). 가득 찬 상태에서 새 데이터를 넣으려면 기존 데이터를 하나 빼야 한다.

삭제 규칙	제거 대상
LRU (Least Recently Used)	가장 **오래전에 사용**된 데이터
LFU (Least Frequently Used)	가장 **적게 사용**된 데이터
FIFO (First In First Out)	**먼저 추가**된 데이터

많은 서비스는 최신 데이터(최신 기사·영상)를 더 자주 조회한다. 오래된 데이터는 캐시에 있어도 쓰이지 않을 가능성이 크므로, 가득 차지 않았더라도 미리 지우는 게 좋다. 이를 위해 **유효 시간(만료 시간, TTL)**을 함께 설정해 일정 시간이 지나면 자동 삭제되게 한다.

4-2. 로컬 캐시와 리모트 캐시

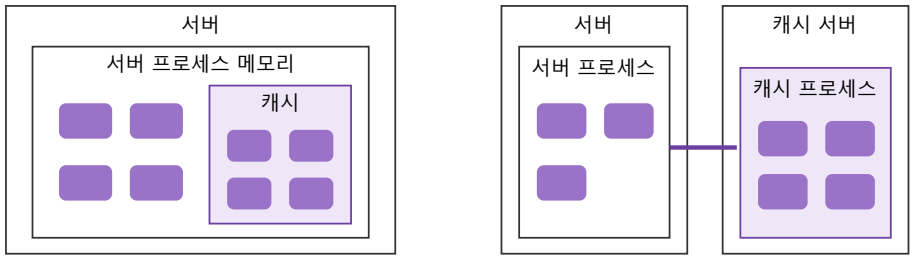


그림 13. 로컬 캐시(왼쪽: 서버 프로세스 메모리 안) vs 리모트 캐시(오른쪽: 별도 프로세스)

알아두기 · "인-메모리 캐시"라는 용어
로컬 캐시를 인-메모리 캐시라고도 부르지만, 리모트 캐시로 많이 쓰는 레디스도 "인-메모리 데이터 저장소"라 불려 헷갈린다. 그래서 책에서는 **로컬 캐시**라는 말을 쓴다.

	로컬 캐시	리모트 캐시
저장 위치	서버 프로세스와 같은 메모리	별도 프로세스(캐시 서버)
대표 기술	Caffeine(자바), go-cache(Go), node-cache(Node.js)	레디스(Redis)
장점	매우 빠름, 외부 연동 없어 구조 단순	크기를 유연하게 확장(레디스 수평 확장), 서버 재시작해도 데이터 유지
단점	프로세스 메모리 한계로 크기 제한, 재시작 시 캐시가 비어 적중률 일시 하락	네트워크 통신이라 상대적으로 느림, 별도 장비·프로세스로 구조 복잡

선택 기준 — 데이터 규모, 변경 빈도, 응답 시간, 처리량

- **작고 잘 안 바뀌는 데이터 → 로컬 캐시**: 예) 홈 화면 최신 공지 목록 (많아야 10개 미만, 몇 시간~며칠 동일)
- **규모가 큰 데이터 → 리모트 캐시**: 예) 대형 쇼핑몰의 개별 상품 정보. 로컬은 메모리가 부족하고, 자주 바뀌면 효율도 떨어진다.
- **배포가 잦은 서비스(하루 여러 번) → 리모트 캐시** 적극 고려: 로컬은 재시작마다 캐시를 다시 쌓아야 해 직후 적중률·응답 시간이 나빠진다.

4-3. 캐시 사전 적재

트래픽이 **순간적으로 급증**하는 패턴이라면 데이터를 미리 캐시에 넣어 두는 것을 고려한다.

가상 사례 · G 앱의 월 요금 안내
사용자 300만 명. 매달 정해진 날 전체 회원에게 요금 안내 푸시 → 받은 사람 중 일부가 바로 앱에서 요금을 조회. 절반이 즉시 확인하면 단시간에 **150만 명**이 몰린다. 이때 개별 요금 정보는 아직 한 번도 조회되지 않아 캐시에 없으므로 **적중률이 0%에 가깝고**, 응답 지연과 DB 부하가 폭증한다.
→ 푸시 발송 **전에** 각 사용자의 요금 정보를 캐시에 넣어 두면 적중률을 99% 가까이 유지하고, 응답 시간 안정·DB 부하 집중 방지 효과를 얻는다.

4-4. 캐시 무효화

원본이 바뀌면 캐시도 **함께 변경하거나 삭제**해야 한다. 그러지 않으면 사용자가 오래된 잘못된 정보를 본다. 무효화 시점은 데이터 특성에 따라 다르게 정한다.

- **민감한 데이터(가격, 게시글 내용)**: 변경 즉시 무효화. 수정했는데 이전 글이 보이면 사용자는 혼란을 겪고, 오류 신고를 하거나 서비스 신뢰도를 낮게 평가한다.
- 변경에 민감한 데이터는 **리모트 캐시**에 둔다. 로컬 캐시는 자기 서버 것만 바뀌므로, A 서버 사용자는 새 가격을, B 서버 사용자는 옛 가격을 보는 문제가 생긴다.
- **민감하지 않고 작은 데이터**: 유효 시간을 둬 주기적으로 갱신해도 된다. 예) 인기 글 목록 TTL 10분 → 10분 주기 갱신, 더 자주 원하면 1분으로.

5. 가비지 컬렉터와 메모리 사용

자바, Go, 파이썬 등은 **가비지 컬렉터(GC)**를 쓴다. 다 쓴 객체를 즉시 지우지 않고, 규칙(예: 힙 사용량이 일정 비율 초과, 일정 주기)에 따라 쓰지 않는 메모리를 찾아 반환한다.

- 장점: 개발자의 메모리 관리 부담 감소, 코드로 메모리를 직접 다루지 않아 보안 이슈도 줄어듦
- 단점: 응답 시간에 영향. 자바는 GC 동안 애플리케이션 실행이 일시 정지 → **Stop-The-World**

메모리를 많이 쓰고 객체가 많을수록 미사용 객체를 찾는 데 오래 걸려 GC 시간이 길어진다. 반대로 예컨대 JVM 최대 힙을 **4GB → 2GB**로 줄이면 검사 대상이 줄어 GC 시간이 짧아질 수 있다. 단, 실제로 4GB 가까이 필요한 앱이면 메모리 부족 에러가 나므로 **실제 사용 패턴에 맞춰** 조정한다.

5-1. 대량 객체 생성 주의

게시글 1개 0.5KB × 10만 개 ≈ 50MB → 동시 100명이 호출하면 ≈ 4.9GB 필요

최대 메모리가 4GB라면 메모리 부족이 발생하고, GC를 돌려도 필요한 양 자체가 4.9GB이므로 부족 상태가 계속된다.

- **조회 범위를 제한**한다: 10년 치 거래 내역 대신 최대 3개월 치만 조회
- 한 번에 조회할 수 있는 **개수**도 트래픽 규모와 메모리 크기에 맞춰 제한

5-2. 스트림 활용

파일 다운로드처럼 큰 데이터를 다룰 때 전체를 메모리에 올린 뒤 응답하지 말고 **스트림**으로 나눠 보낸다.

```
// X 파일 전체를 메모리에 올림 → 30MB 파일을 100명이 받으면 약 3GB 필요
byte[] all = Files.readAllBytes(path);
out.write(all);

// ✓ 8KB 버퍼로 나눠 읽고 쓰기 → 100명이 받아도 약 800KB
try (InputStream in = Files.newInputStream(path)) {
    byte[] buf = new byte[8 * 1024];
    int n;
    while ((n = in.read(buf)) != -1) {
        out.write(buf, 0, n);
    }
}
// Java 9+ 에서는 in.transferTo(out) 한 줄로 같은 동작
```

전체 로딩 방식(≈3GB)과 비교하면 스트림 방식의 메모리 사용량은 0.001%도 되지 않는다.

COLUMN · 대량 데이터를 한 번에 올려 서버가 멈춘 사례

내부 관리 사이트 서버 2대 중 1대가 응답하지 않음 → 모니터링 결과 JVM 최대 메모리를 다 쓰고, 풀 GC가 여러 번 돌아도 포화 상태 지속, 결국 **OOME(Out Of Memory Error)** 발생. 힙 덤프를 분석하니 **엑셀 생성 객체**가 메모리 대부분을 차지. 누군가 수백만 건 엑셀 다운로드를 실행했고, 오래 걸리자 취소 후 재요청 → 같은 데이터가 또 메모리에 올라가 폭증.

조치: ① 엑셀을 메모리가 아닌 로컬 파일에 스트림 형태로 생성 ② DB 조회 결과를 리스트로 한 번에 받지 않고 스트림으로 받아 순차 처리 → 이후 재발 없음.

COLUMN · 디스코드가 러스트로 간 이유

디스코드는 Go로 만든 서버를 러스트로 다시 구현했다. 주된 이유는 **GC로 인한 응답 시간 증가와 CPU 사용률 급등**이었고, GC가 없는 러스트로 성능을 개선했다.

그렇다고 무조건 러스트로 갈 일은 아니다. 익숙한 개발자가 없다면 접어 두자. 대규모 트래픽·고성능이 필수가 아니라면 GC 영향은 대체로 크지 않고, GC 성능도 계속 개선되고 있다. **다른 개선책을 먼저 검토**하고, 그래도 안 될 때 GC 없는 언어를 고려한다.

6. 응답 데이터 압축

응답 시간에는 데이터 **전송 시간**이 포함되고, 전송 시간은 ① **네트워크 속도**와 ② **전송 데이터 크기**에 좌우된다. 서버는 사용자 네트워크 속도는 못 바꾸지만 **데이터 크기는 줄일 수 있다**.

- HTML, CSS, JS, JSON 같은 **텍스트** 응답은 압축 효과가 크다 — gzip으로 **70% 이상** 줄기도 한다 → 전송 시간 단축 = 응답 시간 단축
- 클라우드는 트래픽이 곧 **비용**이므로 압축은 비용 절감에도 직결된다
- 아파치, Nginx 등 웹 서버는 압축 기능이 있어 **설정만 추가**하면 바로 효과를 본다

알아두기 · Accept-Encoding 요청 헤더와 Content-Encoding 응답 헤더

클라이언트는 처리 가능한 압축 알고리즘을 요청 헤더로 알린다.

```
Accept-Encoding: gzip, deflate
```

서버는 그중 자신이 지원하는 방식으로 압축해 보내고, 사용한 알고리즘을 `Content-Encoding` 응답 헤더(예: `Content-Encoding: gzip`)로 알려 준다.

압축 적용 시 고려 사항

- 텍스트 형식만 압축한다. JPEG 이미지, zip 파일처럼 **이미 압축된 데이터**는 다시 압축해도 효과가 없다.
- 웹 서버에서 압축했는데도 실제 응답이 압축되지 않았다면, **방화벽**이 압축을 해제해 보내고 있지 않은지 확인한다.

알아두기 · 트래픽과 비용

응답 크기는 곧 비용이다. AWS 같은 클라우드는 트래픽 양에 비례해 과금하므로, 규모가 커질수록 데이터 크기만 줄여도 절감액이 커진다. IDC 같은 온프레미스도 네트워크 대역폭에 따라 비용이 달라진다. 트래픽이 늘고 있다면 응답 크기를 줄일 수 있는지 확인해 보자.

7. 정적 자원과 브라우저 캐시 / CDN

7-1. 동적 자원 vs 정적 자원

가상의 쇼핑몰: 스프링 부트 + 타임리프로 HTML을 생성하고 JS·CSS도 내려주며, JSON API가 있고, 아파치로 판매자가 올린 상품 이미지를 제공한다. 이 서버의 응답은 두 종류다.

구분	특징	예
동적 자원	요청할 때마다 결과가 달라짐	상품 목록 HTML, 상품 상세 JSON
정적 자원	같은 URL이면 항상 같은 데이터	이미지, JS, CSS

정적 자원은 트래픽에서 큰 비중을 차지한다. 이미지가 많은 쇼핑몰 첫 페이지는 정적 자원이 전체 데이터의 80%에 이르기도 한다(브라우저 개발자 도구로 종류별 용량을 쉽게 확인할 수 있다).

콘텐츠 종류	비율	크기
Image	74.84%	3.0 MB
Script (JS)	19.49%	775.4 KB
HTML	2.07%	82.5 KB
XHR	2.03%	80.6 KB
CSS	1.24%	49.3 KB
Redirect	0.27%	10.6 KB
Error	0.06%	2.6 KB
합계	**100%**	**4.0 MB**

표 1. 한 쇼핑몰 첫 페이지(pingdom 측정) — 이미지와 JS가 약 94%(약 3.7MB)를 차지

7-2. 브라우저 캐시

같은 페이지에 들어갈 때마다 같은 이미지·JS를 다시 받으면 서버엔 불필요한 트래픽과 비용이, 브라우저엔 느린 화면 표시가 생긴다. 예) 상품 상세를 10번 방문하면 같은 파일을 10번 다운로드.

해결책은 **클라이언트(브라우저) 캐시**. HTTP의 `Cache-Control` 또는 `Expires` 헤더로 응답을 일정 시간 저장해 두게 할 수 있다.

```
Cache-Control: max-age=60
```

위처럼 응답하면 브라우저는 파일을 로컬 캐시(메모리·디스크)에 보관하고, **60초 이내** 같은 주소를 요청하면 서버에 가지 않고 로컬 데이터를 쓴다 → 화면이 빨라지고 서버의 네트워크 전송 비용도 준다.

7-3. CDN (Content Delivery Network)

브라우저 캐시는 **브라우저 단위**로 동작하므로, 동시에 많은 사용자가 처음 접속하면 여전히 대량의 이미지·JS·CSS를 전송해야 한다. 그러면 네트워크가 금방 포화되어 응답이 급격히 느려진다. (4차선 도로에 차 몇 대는 잘 지나가지만 수십 대가 몰리면 순식간에 막히는 것과 같다.)

해결책 중 하나가 콘텐츠 전송 전용 네트워크인 **CDN**이다. 대표 서비스: Amazon CloudFront, Akamai, Cloudflare.

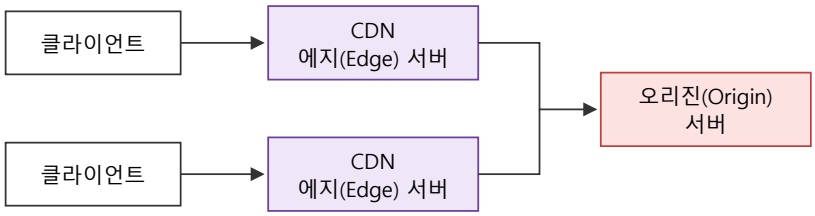


그림 14. 지역별 에지 서버 — 클라이언트는 가까운 에지에 연결해 오리진에 직접 가는 것보다 빠르게 받는다

- 사용자는 CDN이 제공하는 URL로 접근한다. 에지에 콘텐츠가 없으면 **오리진 서버**에서 가져와 제공하고 캐시에 보관 → 이후 같은 요청은 에지가 바로 응답. 같은 파일을 여러 번 요청해도 오리진은 **한 번만** 처리하면 된다.
- 여러 지역에 서버가 있어 사용자 가까이에서 내려받으므로 이미지·JS 로딩과 화면 표시가 빨라진다.
- **장점 3가지**: 오리진 트래픽 감소, 콘텐츠의 빠른 제공, 트래픽 비용 절감 → 서비스가 성장하기 시작하면 도입을 고려하자.

알아두기 · 정적 파일 크기 주의

실수로 30MB짜리 이미지를 올려 비용 청구가 급증한 사례도 있다. 계약 조건상 일정 트래픽을 넘지 못하게 막혀 있으면 용량 초과로 서비스가 불능 상태가 될 수도 있다. 업로드 파일 크기를 신중히 관리하고, 웹 서버에 **크기 제한**을 설정하자(예: 아파치는 일정 크기를 넘는 파일에 에러 코드를 응답하도록 설정 가능) → 네트워크 포화와 과도한 비용을 막는다.

8. 대기 처리

콘서트 예매처럼 짧은 시간(1시간도 안 되는)에 사용자가 폭증했다가 매진과 함께 급감하는 트래픽이 있다.

방법 1. 미리 증설한다 — 그러나 비용 문제

- 클라우드라면 서버는 쉽게 늘리고 줄일 수 있다.
- 하지만 트래픽은 **DB**에도 영향을 주고, DB는 쉽게 증설할 수 없어 예상 트래픽에 맞춰 미리 키워야 한다.
- DB는 한 번 성능을 올리면 다시 줄이기 어렵다 → 전체 시간의 1%도 안 되는 순간을 위해 **고정 비용**이 커진다.

  (예: 30일 중 폭증 1시간 = 약 0.14%. 데이터 저장소는 비싸기 때문에 0.14%를 위해 고정비를 올리는 것은 부담이다.)

방법 2. 받아들일 만큼만 받고 나머지는 대기시킨다

처리량을 무작정 늘리는 대신, 감당 가능한 수준까지만 받아들이고 나머지는 줄을 세운다 — **사용자 대기 시스템**. 은행 창구와 비슷하다: 창구가 모두 차면 나머지는 기다리고, 한 창구가 비면 다음 순번이 들어간다.

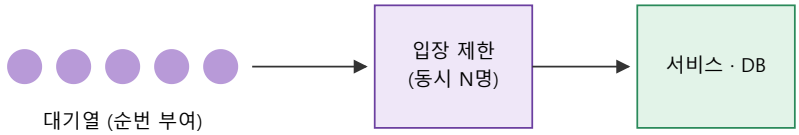


그림 15. 대기 처리 — 동시 입장 수를 제한하고 나머지는 순서대로 대기

이점

- 서버를 증설하지 않고도 서비스를 **안정적으로** 제공한다.
- 새로 고침하면 순번이 뒤로 밀리므로 사용자가 새로 고침을 자제 → **새로 고침 폭증**도 막는다.

사용자는 기다리기 싫어하지만, 과부하로 서비스 자체가 안 되는 것보다는 기다렸다가 결국 쓸 수 있는 편이 낫다.

> **MEMO**
> 대규모 트래픽 아키텍처를 경험해 보고 싶은 마음이 들어도, 개발자는 **비용**을 함께 생각해야 한다. 그런 관점에서 대기 제어는 매우 합리적이다. 직접 구현할 필요 없이 이미 여러 솔루션이 있어 빠르게 도입할 수 있다는 것도 장점이다.

9. 복습 퀴즈

1. 응답 시간을 구성하는 세 구간은?
   → 요청 전송, 서버 처리, 응답 전송
2. TTFB와 TTLB의 차이가 커지는 상황은?
   → 응답 데이터가 크거나(파일 다운로드) 네트워크가 느릴 때
3. 외부 API가 없는 서비스에서 처리 시간의 대부분(70~80%↑)을 차지하는 것은?
   → DB 연동
4. 최대 TPS 5인 서버에 7개 요청이 동시에 오면, 늦게 처리된 2개의 응답 시간은? (요청당 1초)
   → 2초 (대기 1초 + 처리 1초)
5. TPS를 높이는 두 가지 방향은?
   → 동시 처리 수 늘리기, 요청당 처리 시간 줄이기
6. DB가 병목인데 서버를 수평 확장하면?
   → DB 부하가 더 커져 성능이 악화된다 (불에 기름)
7. 로드 밸런서의 정적 분배 방식 두 가지는?
   → 라운드 로빈, IP 해시
8. 풀 크기 5, 쿼리 0.5초일 때 1초 처리량은?
   → 10 (1 ÷ 0.5 × 5)
9. 트래픽이 순간 급증하는 서비스에서 커넥션 풀 최소 크기는 어떻게?
   → 최대 크기와 같게
10. HikariCP 기본 커넥션 대기 시간과 일반 권장 범위는?
    → 기본 30초 / 0.5~3초
11. 최대 유휴 시간은 DB의 비활성 연결 유지 시간보다 길게? 짧게?
    → 짧게
12. 캐시 적중률 공식은?
    → 캐시에 존재한 건수 ÷ 조회 시도 건수
13. LRU, LFU, FIFO가 각각 지우는 대상은?
    → 가장 오래전 사용 / 가장 적게 사용 / 먼저 들어온 것
14. 가격처럼 변경에 민감한 데이터는 로컬·리모트 중 어디에? 이유는?
    → 리모트. 로컬은 다른 서버의 캐시가 갱신되지 않아 서버마다 다른 값을 보여 줄 수 있음
15. 푸시 직후 요청이 몰려 적중률이 0%에 가까워지는 문제의 대책은?
    → 캐시 사전 적재
16. 30MB 파일을 100명이 동시에 다운로드할 때, 전체 로딩과 8KB 스트림의 메모리 차이는?
    → 약 3GB vs 약 800KB
17. gzip 압축 효과가 거의 없는 응답 예시는?
    → JPEG 이미지, zip 파일 등 이미 압축된 데이터
18. `Cache-Control: max-age=60` 의 의미는?
    → 60초 동안 브라우저가 서버에 다시 요청하지 않고 로컬 캐시를 사용
19. CDN 도입의 세 가지 장점은?
    → 오리진 트래픽 감소, 빠른 콘텐츠 제공, 트래픽 비용 절감

20. 짧은 순간의 트래픽 폭증에 DB를 미리 증설하는 대신 쓸 수 있는 방법은?
→ 사용자 대기 처리(대기열 시스템)